PROJECT BUILD GUIDE

Distributed Token-Aware Rate Limiter

How LLM API providers limit requests and tokens across many servers, and a step-by-step plan, design, code and reading list to build your own.

What you'll build	Stack	Time
A gateway that enforces RPM, input-TPM, output-TPM and concurrency limits per org and per model, atomically across many nodes, with reserve-then-refund token accounting.	Python · FastAPI · tiktoken · Redis + Lua · Docker Compose · k6 · Prometheus/Grafana	4 weeks part-time

October 2026

Contents

1. Why this project, and what makes it good . . . 3
2. How OpenAI and Anthropic limit usage . . . 3
3. System architecture . . . 4
4. Choosing the algorithm . . . 5
5. Token accounting: the LLM-specific part . . . 6
6. Making it distributed . . . 7
7. Core code . . . 8
8. API contract: what clients see . . . 10
9. Testing and proving accuracy . . . 10
10. Make it yours: an ML extension . . . 11
11. Four-week build plan . . . 11
12. Trade-offs to talk about in interviews . . . 13
13. Reading list . . . 14

SECTION 01

Why this project, and what makes it good

Every company that sells access to a large language model has to answer the same question thousands of times a second: **is this customer allowed to send this request right now?** A normal API counts requests. An LLM API also has to count **tokens**, and a request's real cost isn't known until the model finishes generating. The limit also has to hold when the customer's traffic is spread across dozens of servers.

That combination is what makes this a strong portfolio project. The simple version is a common tutorial; the version in this guide solves three problems most tutorials skip:

- **Unknown cost.** Output length is unknown at admission time, so you reserve tokens and reconcile afterwards.
- **Several limits at once.** RPM, input TPM, output TPM and concurrency must all pass together, or the request spends nothing (all-or-nothing).
- **Many servers.** Every gateway node must see the same budget, without a race where two nodes spend the same tokens.

What separates a strong version from a weak one

Weak project	Strong project
Token bucket in Redis, counts requests only	Counts input and output tokens separately, with reserve-then-refund
“It works” on one machine	Accuracy measured across 3–5 nodes under load, e.g. “limit 100k TPM, admitted 99.2k”
No failure story	Defined behavior when Redis is slow or down (fail open, local fallback)
No write-up	Design doc explaining every trade-off, plus a Grafana screenshot

SECTION 02

How OpenAI and Anthropic limit usage

Treat the providers' public documentation as your product spec. Details change over time (always check the live docs), but the shape has been stable:

Limit	What it counts	Notes
RPM	Requests per minute	Stops floods of tiny requests
ITPM	Input (prompt) tokens per minute	Counted exactly before the request runs
OTPM	Output (generated) tokens per minute	Unknown up front; estimated from `max_tokens`
TPM	Combined tokens per minute	OpenAI-style single token limit
Concurrency	Requests in flight	Protects GPU capacity from long generations
Spend / tier	Money per month	Moves an org between usage tiers, each with its own limits

Behaviors worth copying

- **Scope:** limits apply per organization and per model. A cheap model and an expensive model have separate buckets.
- **Token bucket:** Anthropic documents that capacity refills continuously up to the maximum, instead of resetting at the top of each minute. That avoids a stampede of retries at :00.
- `max_tokens` **matters:** Anthropic's docs advise lowering `max_tokens` if you hit output limits early, which tells you it's used to estimate output before generation.
- **Prompt caching:** cached input tokens may be counted differently, or not counted, toward input limits. Make this a config flag.
- **Feedback to clients:** HTTP 429 with a `retry-after` header, plus headers saying how much budget is left.

> **KEY IDEA**
> Rate limiting and billing are different systems. The limiter can be approximately right and must be fast. Billing must be exactly right and can be slow. Keep them separate: the limiter emits usage events, and a separate pipeline turns them into invoices.

SECTION 03

System architecture

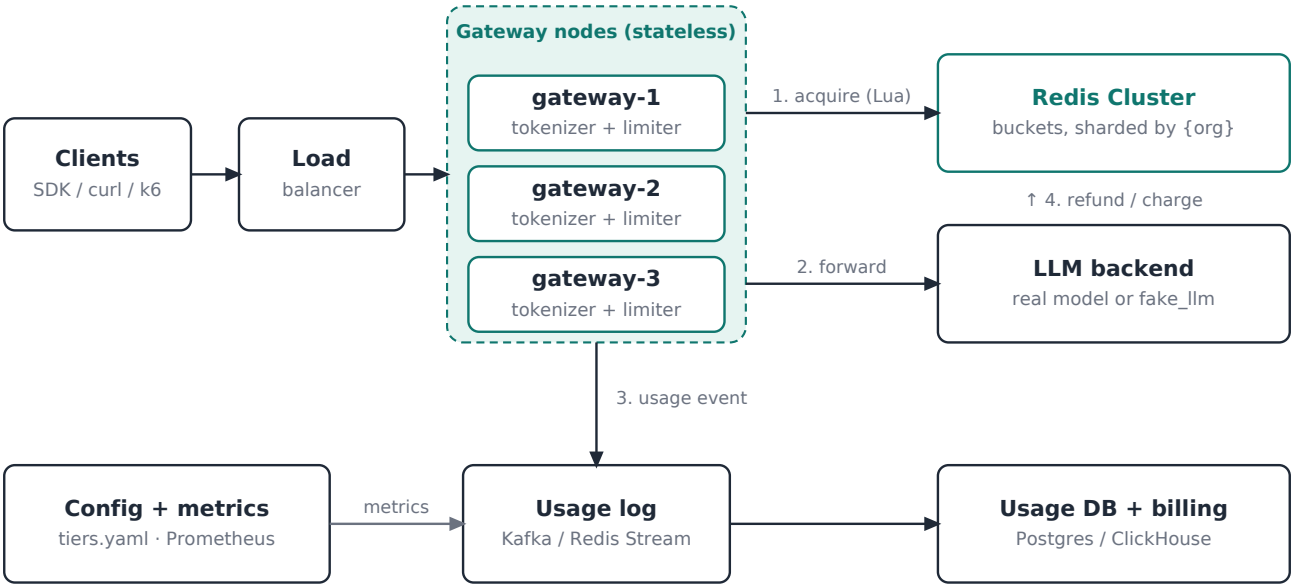


Figure 1. Request path (1→2→4) and usage path (3).

Component	Responsibility
Gateway node	Authenticate the API key → org + tier. Count input tokens. Run the acquire script. Forward to the model. Reconcile actual usage. Set rate-limit headers. Holds no state of its own.
Redis Cluster	Holds all bucket state. Lua scripts make check-and-deduct atomic. Keys use a hash tag `{org_id}` so all of one org's buckets live on the same shard.
LLM backend	The real model, or `fake_llm`: a service that sleeps and returns random token counts, so you can load-test for free.

Component	Responsibility
Usage log	Append-only stream of what each request actually used. Source of truth for billing and analytics.
Config + metrics	Tier definitions (YAML or database), Prometheus counters, Grafana dashboards.

Request lifecycle

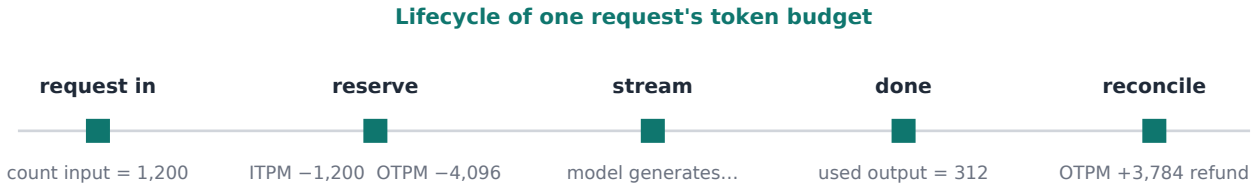


Figure 2. Reserve the worst case, refund the difference.

1. Look up the org and tier from the API key (cache this in memory for ~60 s).
2. Count input tokens with the tokenizer.
3. Acquire a concurrency slot (in-flight request count).
4. Run `acquire.lua`: check RPM (cost 1), ITPM (cost = input), OTPM (cost = `max_tokens`). All pass → deduct all. Any fails → deduct nothing, return 429 with retry-after.
5. Forward to the model. If streaming, count output tokens as chunks arrive.
6. Run `adjust.lua` with `actual − reserved` for each token bucket (usually a refund).
7. Release the concurrency slot, emit a usage event, return the response with headers.

SECTION 04

Choosing the algorithm

Algorithm	How it works	Pros	Cons
Fixed window	Counter per minute, reset at :00	Trivial	2× burst at window edges; retry stampede
Sliding log	Store every request timestamp	Exact	Memory grows with traffic; useless for token weights
Sliding window counter	Weighted mix of current and previous window	Cheap, smooth	Approximate
Token bucket	Bucket refills at rate r up to capacity C; each request removes its cost	Weighted costs, allows bursts up to C, O(1) state	Needs a clock and atomic update
GCRA	Token bucket stored as one “theoretical arrival time”	One number per key	Harder to explain

Use the token bucket. It's what Anthropic documents, it handles weighted costs (1 for a request, 1,200 for a prompt), and it needs only two fields per bucket: `tokens` and `ts` (last update time).

Token bucket math

```
refill:  tokens = min(C, tokens + (now - ts) * r)       # r = limit_per_minute / 60
admit:   if tokens >= cost:  tokens -= cost; allow
         else:               deny, retry_after = (cost - tokens) / r
```

WATCH OUT

Use **one clock**. If each gateway node uses its own system time, clock skew between machines corrupts the refill math. Read the time inside the Lua script with Redis `TIME`, so every decision uses the Redis server's clock.

SECTION 05

Token accounting: the LLM-specific part

Counting input tokens

Count with the same tokenizer the model uses (for practice, `tiktoken`'s `cl100k_base` is fine). Chat formats add a few tokens per message for roles and separators, so add a small constant per message. A slightly high count is safe; a low count lets customers exceed limits.

Reserve, then reconcile

At admission you only know the worst case: `max_tokens`. Reserve that, then correct it when the response finishes:

```
reserved_out = max_tokens              # 4096
actual_out   = usage.output_tokens     # 312
delta        = actual_out - reserved_out   # -3784  -> refund to OTPM bucket
```

Reservation strategy	Effect	When to use
Reserve `max_tokens`	Never overshoots, but rejects too much when users set a high max_tokens	Default; simplest to reason about
Reserve an estimate (e.g. the user's average or P90 output)	Admits more traffic; can overshoot briefly	Advanced; see section 10
Reserve nothing, charge afterwards	Maximum throughput; limit enforced only after the fact	Rarely; easy to abuse

Debt: when the estimate is too low

If you reserve an estimate and the model generates more, `adjust.lua` charges the extra amount. Allow the bucket to go **negative**. The debt is paid back by refill, so the org's next requests wait a little. Over a minute, total usage still matches the limit.

Streaming

For streamed responses, don't wait until the end. Every N chunks (say 50 tokens), charge the tokens produced so far against the reservation. If the client disconnects, reconcile with what was actually produced. This keeps the limiter accurate for long generations and lets you stop runaway streams.

Cached prompt tokens

If you simulate prompt caching, mark part of the input as `cache_read_tokens` and make “count cache reads toward ITPM” a per-model config flag.

SECTION 06

Making it distributed

Atomicity with Lua

The classic bug: node A reads `tokens = 5000`, node B reads `5000`, both deduct 4,000, both admit. Redis runs a Lua script as one atomic step, so read-refill-check-deduct for all buckets happens with nothing in between. It's also one network round trip.

Key design

Redis keys — the {org_42} hash tag puts all of an org's keys on one cluster shard

```
rl:{org_42}:large-model:rpm        hash  {tokens, ts}
rl:{org_42}:large-model:itpm       hash  {tokens, ts}
rl:{org_42}:large-model:otpm       hash  {tokens, ts}
rl:{org_42}:inflight               zset  request_id -> lease expiry
```

Redis Cluster only lets one script touch multiple keys when they live on the same shard. The `{...}` hash tag guarantees that. Each key gets an EXPIRE so idle orgs disappear (an idle bucket refills to full anyway).

Failure handling

Situation	Recommended behavior
Redis timeout (>5–10 ms budget)	**Fail open**: admit, log, count a metric, alert. A limiter outage shouldn't become an API outage.
Redis down for long	Fall back to an in-memory limiter per node with limit ÷ number of nodes.
Gateway crashes mid-request	Reservation stays spent (safe). Concurrency slot expires via its lease TTL.
Hot org (huge traffic)	Local token leasing (below), or dedicate a shard to that org.

Scaling further: local token leasing

At very high request rates, one Redis round trip per request becomes the bottleneck. Instead, each node **leases** a block of tokens (say 2,000) from Redis, spends it from local memory, and fetches more when it runs out. Unused leases are returned or expire. Worst-case overshoot is **nodes × lease size**, so make the lease size adaptive: small for low-limit orgs, large for big ones. Cloudflare's post (reading list) covers this style of thinking.

SECTION 07

Core code

Working sketches to start from. They are deliberately compact: fill in the helpers, add tests, and keep the Lua scripts in their own files so you can unit-test them against a real Redis.

lua/acquire.lua

```
-- acquire.lua: check-and-deduct across N token buckets, all-or-nothing.
-- KEYS[i] = bucket key            ARGV = cap_1, rate_1, cost_1, cap_2, rate_2, cost_2, ...
-- rate = tokens per second. Needs Redis >= 5 (TIME inside scripts).
local t = redis.call('TIME')
local now = tonumber(t[1]) + tonumber(t[2]) / 1e6        -- one clock: Redis server time
local new, remaining = {}, {}
for i = 1, #KEYS do
  local cap  = tonumber(ARGV[(i-1)*3 + 1])
  local rate = tonumber(ARGV[(i-1)*3 + 2])
  local cost = tonumber(ARGV[(i-1)*3 + 3])
  if cost > cap then return {0, i, -1} end                -- can never fit: don't tell them to retry
  local b = redis.call('HMGET', KEYS[i], 'tokens', 'ts')
  local tokens = tonumber(b[1]) or cap
  local ts     = tonumber(b[2]) or now
  tokens = math.min(cap, tokens + math.max(0, now - ts) * rate)
  if tokens < cost then
    return {0, i, math.ceil((cost - tokens) / rate * 1000)}   -- denied: bucket i, retry after ms
  end
  new[i] = tokens - cost
  remaining[i] = math.floor(new[i])
end
for i = 1, #KEYS do                                        -- all passed: commit every bucket
  local cap  = tonumber(ARGV[(i-1)*3 + 1])
  local rate = tonumber(ARGV[(i-1)*3 + 2])
  redis.call('HSET', KEYS[i], 'tokens', tostring(new[i]), 'ts', tostring(now))
  redis.call('EXPIRE', KEYS[i], math.ceil(cap / rate) + 60)
end
return {1, 0, 0, unpack(remaining)}
```

lua/adjust.lua

```
-- adjust.lua: correct a bucket after the response. delta < 0 refunds, delta > 0 charges extra.
-- KEYS[1] = bucket key            ARGV = cap, rate, delta
local t = redis.call('TIME')
local now = tonumber(t[1]) + tonumber(t[2]) / 1e6
local cap, rate, delta = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(b[1]) or cap
local ts     = tonumber(b[2]) or now
tokens = math.min(cap, tokens + math.max(0, now - ts) * rate)
tokens = math.min(cap, tokens - delta)        -- may go negative: debt is repaid by refill
redis.call('HSET', KEYS[1], 'tokens', tostring(tokens), 'ts', tostring(now))
redis.call('EXPIRE', KEYS[1], math.ceil(cap / rate) + 60)
return math.floor(tokens)
```

lua/concurrency.lua

```
-- concurrency.lua: take an in-flight slot with a lease, so crashed requests can't leak slots.
-- KEYS[1] = rl:{org}:inflight     ARGV = limit, request_id, lease_seconds
local now = tonumber(redis.call('TIME')[1])
redis.call('ZREMRANGEBYSCORE', KEYS[1], '-inf', now)     -- drop expired leases
if redis.call('ZCARD', KEYS[1]) >= tonumber(ARGV[1]) then return 0 end
redis.call('ZADD', KEYS[1], now + tonumber(ARGV[3]), ARGV[2])
redis.call('EXPIRE', KEYS[1], tonumber(ARGV[3]) + 60)
return 1
-- release with: ZREM rl:{org}:inflight <request_id>
```

gateway/app.py

```
# gateway/app.py  (sketch: lookup_org, call_model, emit_usage_event, too_many, rate_headers are yours)
import yaml, tiktoken, redis.asyncio as redis
from fastapi import FastAPI, Request, Header
from fastapi.responses import JSONResponse

app = FastAPI()
r = redis.Redis(host="redis", decode_responses=True)
enc = tiktoken.get_encoding("cl100k_base")
TIERS = yaml.safe_load(open("config/tiers.yaml"))
ACQUIRE = r.register_script(open("lua/acquire.lua").read())
ADJUST = r.register_script(open("lua/adjust.lua").read())

def count_input_tokens(messages):
    return sum(len(enc.encode(m["content"])) + 4 for m in messages) + 2   # +4/msg format overhead

@app.post("/v1/messages")
async def messages(req: Request, x_api_key: str = Header()):
    body = await req.json()
    org = await lookup_org(x_api_key)                       # api key -> org id + tier (cached)
    model, max_tokens = body["model"], body.get("max_tokens", 1024)
    lim = TIERS[org.tier][model]
    n_in = count_input_tokens(body["messages"])

    keys = [f"rl:{{{org.id}}}:{model}:{d}" for d in ("rpm", "itpm", "otpm")]
    args = [lim["rpm"],  lim["rpm"] / 60,  1,
            lim["itpm"], lim["itpm"] / 60, n_in,
            lim["otpm"], lim["otpm"] / 60, max_tokens]
    try:
        res = await ACQUIRE(keys=keys, args=args)
    except redis.RedisError:
        res = [1, 0, 0, -1, -1, -1]                         # fail open: log + metric + alert
    if res[0] == 0:
        return too_many(which=keys[res[1] - 1], retry_ms=res[2])   # 429, or 400 if retry_ms == -1

    resp = await call_model(body)                           # real model or fake_llm
    used_in, used_out = resp["usage"]["input_tokens"], resp["usage"]["output_tokens"]
    await ADJUST(keys=[keys[1]], args=[lim["itpm"], lim["itpm"] / 60, used_in - n_in])
    await ADJUST(keys=[keys[2]], args=[lim["otpm"], lim["otpm"] / 60, used_out - max_tokens])
    await emit_usage_event(org.id, model, used_in, used_out)  # exact usage -> billing pipeline
    return JSONResponse(resp, headers=rate_headers(lim, res))
```

fake_llm/app.py

```
# fake_llm/app.py: stands in for a real model so load tests cost nothing
import asyncio, random, tiktoken
from fastapi import FastAPI
app, enc = FastAPI(), tiktoken.get_encoding("cl100k_base")

@app.post("/v1/messages")
async def fake(body: dict):
    n_in = sum(len(enc.encode(m["content"])) + 4 for m in body["messages"]) + 2
    n_out = min(body.get("max_tokens", 1024), int(random.lognormvariate(5.5, 0.8)))  # skewed lengths
    await asyncio.sleep(0.2 + n_out / 100)                  # ~100 tokens/s "generation"
    return {"content": [{"type": "text", "text": "tok " * n_out}],
            "usage": {"input_tokens": n_in, "output_tokens": n_out}}
```

config/tiers.yaml (made-up numbers; shape modeled on public provider tiers)

```
free:
  small-model: {rpm: 5,    itpm: 10000,   otpm: 2000,   concurrency: 2}
tier1:
  small-model: {rpm: 50,   itpm: 50000,   otpm: 10000,  concurrency: 10}
  large-model: {rpm: 50,   itpm: 30000,   otpm: 8000,   concurrency: 5}
tier2:
  small-model: {rpm: 1000, itpm: 450000,  otpm: 90000,  concurrency: 50}
  large-model: {rpm: 1000, itpm: 200000,  otpm: 40000,  concurrency: 25}
```

SECTION 08

API contract: what clients see

Header	Meaning
`x-ratelimit-limit-requests`	RPM limit for this org and model
`x-ratelimit-remaining-requests`	Requests left in the bucket right now
`x-ratelimit-limit-tokens` / `-remaining-tokens`	Same for tokens (add input/output variants if you split them)
`x-ratelimit-reset-tokens`	Time until the token bucket is full again
`retry-after`	Seconds to wait, only on a 429

OpenAI uses `x-ratelimit-*` names; Anthropic uses `anthropic-ratelimit-*` with separate input and output token headers. Pick one style and document it.

Example 429 response — say which limit was hit

```
HTTP/1.1 429 Too Many Requests
retry-after: 7
content-type: application/json

{"type": "error",
 "error": {"type": "rate_limit_error",
           "message": "OTPM limit (8000) exceeded for large-model. Retry in 7s."}}
```

Also return **400**, not 429, when a single request can never fit (for example `max_tokens` larger than the whole OTPM bucket). Telling a client to retry something that can never succeed creates an infinite retry loop.

SECTION 09

Testing and proving accuracy

Test layers

Layer	What to test
Unit	Refill math with a fake clock; cost > capacity; refund never exceeds capacity; debt goes negative and recovers.
Script	Run the Lua scripts against a real Redis (testcontainers or docker). All-or-nothing: if OTPM fails, RPM and ITPM are untouched.
Race	1,000 concurrent requests from 5 gateway nodes against one org. Assert admitted tokens ≤ C + r × elapsed.
Chaos	Kill Redis mid-test → requests still succeed (fail open) and an alert metric fires. Kill a gateway → its concurrency slots expire.
Load	k6 or Locust at increasing request rates; record the numbers below.

Numbers to put in your README

Metric	Definition	Good target
Accuracy	admitted ÷ (C + r × test duration), per bucket	97–100%, never above ~101%
Added latency	Gateway p50 / p99 overhead vs. calling fake_llm directly	p99 < 5 ms
Redis ops per request	Round trips per request	2 (acquire + adjust); < 1 with leasing
Over-reservation	Average (reserved − used) output tokens	Show how estimates shrink it
Throughput	Requests per second the cluster sustains	Report with node count

KEY IDEA

One chart beats a page of text: plot **allowed tokens per second vs. time** for one org under overload. A flat line at exactly the limit, while offered load doubles, is your proof the limiter works.

SECTION 10

Make it yours: an ML extension

Reserving `max_tokens` wastes capacity: users often set 4,096 and get 300. A model that **predicts output length** lets you reserve a realistic amount and admit more traffic, while debt-and-refill handles the misses. This connects systems engineering with ML, and very few portfolio projects do it.

- **Features:** input token count, model, `max_tokens`, the user's recent output lengths, time of day, whether the prompt asks for code or a list.
- **Model:** start with gradient boosting (LightGBM). Predict the P90 output length with quantile loss, so you under-reserve only about 10% of the time.
- **Data:** your usage log. With fake_llm, generate output lengths that depend on the prompt so there's a signal to learn.
- **Evaluation:** compare false rejections, overshoot, and goodput (useful tokens served) for `max_tokens` reservation vs. predicted reservation, at the same limits.

SECTION 11

Four-week build plan

Week 1 — single node

- ☐ FastAPI gateway + fake_llm service
- ☐ In-memory token bucket for RPM, ITPM and OTPM
- ☐ Input token counting with tiktoken
- ☐ Reserve-then-refund flow, 429 + retry-after, rate-limit headers
- ☐ Unit tests with a fake clock

Week 2 — distributed

- ☐ Move state to Redis with acquire.lua and adjust.lua
- ☐ docker-compose with 3+ gateways behind nginx
- ☐ Race test: concurrent requests across nodes never exceed the limit
- ☐ Concurrency limiter with leases
- ☐ Tier config file, per-org and per-model limits

Week 3 — production behavior

- ☐ Streaming responses with incremental charging
- ☐ Fail-open path + in-memory fallback
- ☐ Usage event stream + simple usage table
- ☐ Prometheus metrics + Grafana dashboard
- ☐ k6 load tests; record accuracy and latency

Week 4 — stand out

- ☐ Local token leasing; measure Redis ops/request and overshoot
- ☐ Optional: output-length predictor (section 10)
- ☐ README with architecture diagram, results table, overload chart
- ☐ DESIGN.md explaining every trade-off in section 12

Suggested repository layout

```
llm-rate-limiter/
├── gateway/            app.py, limiter.py, tokenizer.py, headers.py
├── lua/                acquire.lua, adjust.lua, concurrency.lua
├── fake_llm/           app.py
├── config/             tiers.yaml
├── usage/              consumer.py  (usage events -> table)
├── predictor/          train.py, model.py  (optional ML extension)
├── loadtest/           k6_overload.js, k6_mixed_tiers.js
├── tests/              test_bucket_math.py, test_lua_atomic.py, test_race.py, test_failopen.py
├── observability/      prometheus.yml, grafana_dashboard.json
├── docker-compose.yml  redis + 3 gateways + nginx + fake_llm + prometheus + grafana
├── README.md           results first, then how to run
└── DESIGN.md
```

SECTION 12

Trade-offs to talk about in interviews

Decision	Option A	Option B	What to say
Output reservation	Reserve max_tokens	Reserve an estimate	A never overshoots but wastes capacity; B needs debt handling
Algorithm	Token bucket	Sliding window	Bucket allows controlled bursts and weighted costs
Redis failure	Fail open	Fail closed	Availability over strictness, with alerting
Hot path	Redis per request	Local leasing	Leasing cuts latency, overshoot ≤ nodes × lease
Clock	Gateway clocks	Redis TIME	One clock removes skew bugs
Billing	Use limiter counters	Separate usage log	Limiter is approximate; billing must be exact

SECTION 13

Reading list

Read them roughly in this order. Each one covers part of this project. Provider limits and numbers change, so check the live docs before quoting any figure.

Foundations

- **Stripe — Scaling your API with rate limiters**
  `https://stripe.com/blog/rate-limiters`
  Four limiter types in production, including concurrency limits and load shedding.
- **Figma — An alternative approach to rate limiting**
  `https://www.figma.com/blog/an-alternative-approach-to-rate-limiting/`
  Clear comparison of fixed window, sliding window and token bucket.
- **System Design Interview, Vol. 1 (Alex Xu) — rate limiter chapter**
  `https://bytebytego.com`
  Structured overview of algorithms and distributed race conditions.

Distributed

- **GitHub — Sharded, replicated rate limiter in Redis**
  `https://github.blog/engineering/how-we-scaled-github-api-sharded-replicated-rate-limiter-redis/`
  Lua for atomicity, sharding keys, key expiry instead of reset timestamps.
- **Cloudflare — Rate limiting capable of scaling to millions of domains**
  `https://blog.cloudflare.com/counting-things-a-lot-of-different-things/`
  Staying accurate across many servers without a central call per request.
- **Brandur Leach — Rate limiting, cells, and GCRA**
  `https://brandur.org/rate-limiting`
  GCRA: a token bucket stored as a single timestamp.

LLM-specific

- **Anthropic — API rate limits**
  `https://docs.claude.com/en/api/rate-limits`
  Token bucket, RPM / ITPM / OTPM per model, usage tiers, response headers.
- **OpenAI — Rate limits guide**
  `https://platform.openai.com/docs/guides/rate-limits`
  RPM/TPM limits, tiers, and the x-ratelimit-* headers.
- **Envoy AI Gateway — Usage-based rate limiting**
  `https://aigateway.envoyproxy.io/docs/capabilities/traffic/usage-based-ratelimiting`
  Open-source token-based limiting on Redis; weights output vs. input tokens; quotas vs. rate limits.

Code to read

- **envoyproxy/ratelimit (Go)**
  `https://github.com/envoyproxy/ratelimit`
  A production distributed rate-limit service.
- **brandur/redis-cell (Rust)**
  `https://github.com/brandur/redis-cell`
  GCRA as a Redis module.
- **BerriAI/litellm (Python)**
  `https://github.com/BerriAI/litellm`
  LLM proxy with per-key TPM/RPM limits — closest to your gateway.